FormaComposition User Manual

The Intervals Engine — Generative MIDI Composition System

---

1. What FormaComposition Does

FormaComposition is a generative MIDI composition engine. You describe a piece of music in two JSON files — a theme and a composition — and the engine produces a multi-track MIDI file with harmony, melody, bass, and optional counterpoint. The MIDI imports into a DAW (Logic Pro is the primary target), where you assign instruments and play or sing over the generated bed.

The engine is not a sequencer. It does not record or replay performances. It generates new musical material every time, guided by a set of musical constraints you define: key, mode, chord progressions, density, melodic behavior, motif development, bass style, and emotional arc. The output is a starting point — a composed bed that has structure, breathability, and enough direction to feel like it goes somewhere, while leaving room for a human performer to inhabit the piece.

The workflow:

1. Write a theme JSON (musical identity: key, mode, tempo, motif)
2. Write a composition JSON (structure: sections, progressions, behaviors)
3. Run the engine to generate a MIDI file
4. Import into Logic Pro, assign instruments (Arturia V Collection or any VST/AU)
5. Play over it — keyboard, guitar, voice
6. Iterate: tweak JSON, regenerate, listen

---

2. Musical Theory Behind the Engine

2.1 Modes

FormaComposition works in the seven diatonic modes. Each mode is a rotation of the major scale, producing a different pattern of whole and half steps and therefore a different emotional character.

ionian is the standard major scale. Bright, resolved, familiar. The key of "happy." In G ionian, every chord resolves cleanly and progressions sound warm and complete. This is the mode of the Rebecca compositions.

dorian is the natural minor scale with a raised 6th. It has a warm, slightly melancholic quality — darker than major but not as bleak as pure minor. D dorian is the key of the Evening Water theme. The raised 6th gives the ii chord a major quality that adds unexpected warmth to minor-key progressions.

phrygian has a flattened 2nd, giving it a Spanish or Middle Eastern flavor. The half-step between the first and second degrees creates tension on the very first move away from the root.

lydian has a raised 4th — the most "floating" mode. The raised 4th eliminates the perfect 4th's pull toward the root, so progressions feel suspended and luminous. F lydian is the key of the Open Sky theme. The I-II progression in lydian (F major to G major) is the signature sound — two major chords a whole step apart, neither one feeling like "home."

mixolydian is the major scale with a flattened 7th. It has a bluesy, rock quality. The dominant 7th chord on the tonic is built in.

aeolian is the natural minor scale. Darker and more somber than dorian, without the raised 6th's warmth.

locrian has a flattened 2nd and flattened 5th. The diminished chord on the tonic makes it inherently unstable. Rarely used as a primary mode, but available for experimental work.

2.2 Chord Progressions and Roman Numerals

Progressions are written in Roman numeral notation relative to the mode. Uppercase numerals (I, IV, V) indicate major chords; lowercase (i, iv, v) indicate minor chords. The engine resolves the correct chord quality automatically based on the mode and scale degree.

In G ionian: I = G major, ii = A minor, iii = B minor, IV = C major, V = D major, vi = E minor, vii° = F# diminished.

In D dorian: i = D minor, II = E major (because of the raised 6th), III = F major, iv = G minor, v = A minor, VI = Bb major, VII = C major.

You can override the natural quality by appending a suffix: "iim7" forces a minor 7th chord on the second degree, "Vmaj9" forces a major 9th on the fifth degree. Available quality suffixes: maj, min, dim, aug, maj7, m7, 7, dim7, m9, maj9, 9, m11, 11.

How many tones actually sound depends on density. At "sparse," only triads are voiced. At "medium," seventh chords are added. At "full," the engine extends to ninths and elevenths.

2.3 Voice Leading

The engine automatically handles voice leading — the smooth movement of individual chord tones from one chord to the next. When resolving a progression, it evaluates all possible inversions of each chord and chooses the one that minimizes total voice movement from the previous chord while keeping notes within the target register (C3 to C5 for harmony). This

means chords connect smoothly without large jumps, even when the Roman numeral progression moves through distant scale degrees.

2.4 Motifs and Development

A motif is the melodic DNA of a piece — a short sequence of intervals (semitone steps between notes) paired with a rhythmic profile (note durations in beats). The motif is defined once in the theme and developed throughout the piece using Bach-style transforms.

Available transforms:

- **inversion**: negate all intervals (mirror the shape — what went up now goes down)
- **retrograde**: reverse the interval sequence (play the motif backwards)
- **retrograde_inversion**: reverse then negate (Bach's fourth transform)
- **augmentation**: double all note durations (stretch the motif in time)
- **diminution**: halve all note durations (compress the motif)
- **transpose_up**: shift all intervals up by 2 semitones
- **transpose_down**: shift all intervals down by 2 semitones
- **shuffle**: randomly reorder intervals
- **expand**: scale intervals by 1.5x (wider leaps)
- **compress**: scale intervals by 0.5x (smaller steps)

The "develop" melody behavior selects a random transform from the motif's transform_pool each time it generates melody over a chord, then maps the transformed intervals onto scale and chord tones. This gives the piece coherence (everything derives from the same melodic seed) with variety (each statement of the motif sounds different).

2.5 Prosody — Speech to Motif

Instead of hand-crafting a motif, you can provide a text phrase and let the prosody engine derive the motif from the natural stress and rhythm of speech.

When you say "Rebecca" naturally, you're already making music: re-BEC-ca is short-LONG-short, with pitch rising on the stressed syllable. The prosody engine analyzes syllable stress (using the CMU Pronouncing Dictionary when available, falling back to heuristic analysis), vowel duration, and pitch contour, then maps these to musical intervals.

When harmonic context is provided (key, mode, chord, section properties), the prosody engine goes further: instead of mapping stress to raw semitone intervals, it maps to actual chord tones and extensions — root, third, fifth, seventh, ninth — chosen according to how much harmonic tension the section allows.

The tension system has five levels, from stable to expressive:

- **STABLE**: root and fifth (always safe, used on stressed downbeats and phrase endings)
- **WARM**: third, sixth, major 7th (adds color without tension)
- **COLOR**: ninth, thirteenth (rich, slightly yearning)
- **TENSION**: minor 7th, eleventh (pulls toward resolution)
- **PASSING**: chromatic neighbors (very brief, unstressed syllables only)

Section properties control how far up this hierarchy the engine is allowed to go:

- The **arc** sets the tension ceiling (swell allows more than fade_in)
- The **melody** behavior sets how often tension tones are used (develop > lyrical > sparse)
- The **density** controls the passing tone budget

Voice leading rules are enforced: tension tones resolve by step, phrases always end on stable tones, and consecutive large leaps in the same direction are avoided.

Because prosody is section-aware, the same phrase generates a different motif in each section. "Rebecca" in a sparse fade_in section produces stable, simple intervals; in a full-density swell/develop section, the same word carries more harmonic color and tension. The word is "re-sung" differently in each emotional context.

2.6 Counterpoint

The counterpoint system generates a second melodic voice against the primary melody, following classical voice-leading rules. Two species are available:

First species (note-against-note): strict consonance rules, one counterpoint note per melody note. Every interval must be consonant. Parallel perfect fifths and octaves are forbidden. Contrary motion is preferred. Phrases end on perfect consonances.

Free species: mixed rhythms, passing tones, suspensions. More musical and organic. Dissonance is allowed on weak beats as passing tones (configurable). Occasional rests create breathing space.

The scoring algorithm evaluates every possible counterpoint note at each step, penalizing hard rule violations (parallel fifths, dissonance on strong beats), rewarding contrary motion and stepwise movement, and selecting the best candidate. The result is a voice that sounds composed rather than random — it follows the melody intelligently, creating musical tension and resolution.

---

3. Project Structure

```
FormaComposition/
├── compositions/          # Theme and piece JSON files
|   ├── theme_rebecca.json
|   ├── theme_rebecca_prosody.json
|   ├── theme_evening_water.json
|   ├── theme_open_sky.json
|   ├── rebecca.json
|   ├── still_cove.json
|   └── light_on_still_water.json
├── forma/                 # Engine source code
|   ├── main.py            # CLI entry point, validation, info display
|   └── intervals/
|       ├── core/
|       |   └── generator.py   # Assembles all voices into MIDI
|       └── music/
|           ├── harmony.py     # Chord resolution and voice leading
|           ├── melody.py      # Melodic line generation
|           ├── bass.py        # Bass line generation
|           ├── counterpoint.py # Counterpoint voice generation
|           ├── rhythm.py      # Rhythmic pattern generation
|           ├── motif.py       # Motif definition and transforms
|           └── prosody.py     # Speech-to-motif conversion
├── output/                # Generated MIDI files
└── requirements.txt       # Python dependencies
```

---

4. Installation

Requirements

- Python 3.10+
- A virtual environment is recommended

Setup

```bash
```

```
cd FormaComposition
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Dependencies

mido — MIDI file reading and writing. Required.

numpy — Numerical operations. Required.

pronouncing — CMU Pronouncing Dictionary for accurate syllable stress analysis in prosody.py. Optional. If not installed, prosody falls back to heuristic vowel-counting for syllable detection and default stress patterns. The fallback works well for common English words but may be less accurate for unusual words or proper nouns not in the CMU dictionary.

---

5. Running the Engine

All commands are run from the `forma/` directory.

Generate a MIDI file

```bash
python main.py ../compositions/theme_rebecca.json ../compositions/rebecca.json
```

Output is written to `output/<piece_name>.mid` by default.

Specify output path

```bash
python main.py ../compositions/theme_rebecca.json ../compositions/rebecca.json \
    --output ../output/rebecca_v2.mid
```

Batch generation (multiple pieces from one theme)

```bash
python main.py ../compositions/theme_evening_water.json \
    ../compositions/still_cove.json \
    ../compositions/light_on_still_water.json \
    --outdir ../output/album/
```

Display piece info without generating

```bash
python main.py ../compositions/theme_rebecca.json ../compositions/rebecca.json --inf
```

This prints key, mode, tempo, motif analysis, section layout, total duration, and (if using prosody) the stress pattern and syllable count.

---

6. MIDI Track Layout

The engine produces a Type 1 MIDI file with the following tracks:

Track	Voice	MIDI Channel	Purpose
0	Metadata	—	Tempo, time signature, piece title
1	Melody	0	Primary melodic voice
2	Counterpoint	1	Secondary melodic voice (only if any section defines it)
3	Harmony	2	Chord voicings
4	Bass	3	Bass line

No GM program_change messages are written. Assign instruments in your DAW. The track order puts melody first, which makes it the default focus when importing into Logic Pro.

---

7. Theme JSON Reference

The theme defines the musical identity of a piece — what key it's in, what mode, what tempo range, and what melodic material to work with. Multiple compositions can share a single theme.

Complete Structure

```json
```

```
{
  "theme": {
    "name": "Rebecca",
    "key": "G",
    "mode": "ionian",
    "tempo": { "min": 48, "max": 66 },
    "motif": {
      "name": "rebecca",
      "intervals": [2, 1, -1, 3, -2],
      "rhythm": [1.0, 0.5, 0.5, 1.0, 1.0],
      "transform_pool": ["inversion", "augmentation", "retrograde"]
    },
    "palette": {
      "harmony": "strings",
      "melody": "warm_pad",
      "bass": "cello",
      "counterpoint": "strings"
    },
    "notes": "Free-text notes for your own reference."
  }
}
```

Parameter Reference

name (string) — Human-readable name for the theme. Used in info display and as a default motif name for prosody. Not used by the engine for generation.

key (string, required) — The tonal center. Any note name: C, C#, D, D#, E, F, F#, G, G#, A, A#, B. Flats are also accepted: Db, Eb, Gb, Ab, Bb. The engine normalizes flats to their enharmonic sharp equivalents internally.

mode (string, required) — The modal scale to use. One of: ionian, dorian, phrygian, lydian, mixolydian, aeolian, locrian. This determines which chords are natural at each scale degree and what scale tones are available for melody and bass.

tempo (object, required) — Tempo range in BPM. Must have "min" and "max" keys. The composition's tempo must fall within this range (a warning is issued if it doesn't, but generation proceeds). If the composition doesn't specify a tempo, the engine uses the midpoint of the range.

- **tempo.min** (integer) — Minimum BPM
- **tempo.max** (integer) — Maximum BPM

motif (object, optional) — The melodic seed for the piece. If present, this takes precedence over the "phrase" field. If neither "motif" nor "phrase" is defined, melody generation uses purely

generative or lyrical algorithms without motif development.

- **motif.name** (string) — Label for identification. Appears in info display.
- **motif.intervals** (array of integers) — Semitone steps between successive notes. Positive values move up, negative values move down. For example, [2, -1, 3, -2] means: up a whole step, down a half step, up a minor third, down a whole step. The number of intervals determines the motif's note count (intervals + 1 notes).
- **motif.rhythm** (array of floats) — Duration in beats for each interval transition. 1.0 = quarter note, 0.5 = eighth note, 2.0 = half note. Should be the same length as intervals. If shorter, it is repeated cyclically to match; if longer, it is trimmed.
- **motif.transform_pool** (array of strings) — Which transforms the "develop" melody behavior is allowed to apply. Choose from: inversion, retrograde, retrograde_inversion, augmentation, diminution, transpose_up, transpose_down, shuffle, expand, compress. A smaller pool produces more recognizable variations; a larger pool produces more variety.

phrase (string, optional) — A text phrase to convert into a motif via prosody analysis. Used only when "motif" is not present. The engine analyzes syllable stress and maps it to musical intervals, producing a section-aware motif that adapts to each section's harmonic context and tension profile. See the Prosody section for details.

If both "motif" and "phrase" are present, the engine issues a warning and uses the explicit motif.

palette (object, optional) — Instrument suggestions for DAW setup. Not used by the engine — purely for your reference when assigning instruments in Logic Pro.

- **palette.harmony** — Suggested instrument for the harmony track
- **palette.melody** — Suggested instrument for the melody track
- **palette.bass** — Suggested instrument for the bass track
- **palette.counterpoint** — Suggested instrument for the counterpoint track

notes (string, optional) — Free-text notes. Not used by the engine.

Designing Motifs

A good motif for ambient/new-age work is short (3-6 intervals), singable, and has a mix of stepwise motion and one expressive leap.

The Rebecca motif [2, 1, -1, 3, -2] is a gentle stepwise rise (up 2, up 1) with a small dip (-1), then one expressive leap (+3) that gives it emotional weight, followed by a settling step (-2). The rhythm [1.0, 0.5, 0.5, 1.0, 1.0] starts steady, quickens through the middle, and broadens at the end — it breathes.

For a more angular, floating motif, the Open Sky uses [6, -2, 1, -3] — a large opening leap (tritone) that immediately pulls back. This works in lydian because the raised 4th makes the tritone feel suspended rather than tense.

Keep the transform_pool conservative for emotional pieces (inversion, augmentation, retrograde). Add more aggressive transforms (shuffle, expand, compress) for experimental or evolving textures.

---

8. Composition JSON Reference

The composition defines the structure of a specific piece: its sections, chord progressions, melodic behavior, bass style, and emotional arc. It references a theme for its key, mode, and motif.

Complete Structure

```
json
```

```json
{
  "piece": {
    "title": "Rebecca",
    "theme": "Rebecca",
    "tempo": 56,
    "notes": "Free-text description of the piece.",
    "logic_instruments": { },
    "sections": [
      {
        "name": "approach",
        "bars": 12,
        "progression": ["I", "V", "vi", "IV"],
        "density": "sparse",
        "melody": "sparse",
        "bass_style": "pedal",
        "arc": "fade_in",
        "beats_per_bar": 4,
        "counterpoint": {
          "species": "free",
          "register": "below",
          "dissonance": "passing",
          "velocity": 52
        },
        "notes": "Section-specific notes."
      }
    ]
  }
}
```

Top-Level Parameters

title (string) — The piece name. Written as a MIDI meta-event in Track 0 and appears in the DAW.

theme (string, optional) — Name of the referenced theme. For your reference only — the engine receives the theme file as a separate command-line argument.

tempo (integer, optional) — BPM for this piece. Should fall within the theme's tempo range. If not specified, the engine uses the midpoint of the theme's range.

notes (string, optional) — Free-text notes. Not used by the engine.

logic_instruments (object, optional) — DAW instrument assignments and notes. Not used by the engine. Structure is freeform — use whatever keys make sense for your workflow.

logic_session (string, optional) — Path to Logic Pro session file. For your reference.

Section Parameters

Each section is one emotional or structural block within the piece. Sections play in sequence with their beat offsets calculated automatically.

name (string) — Section label. Used in info display and logging.

bars (integer, required) — Total number of bars in this section. Bars are distributed evenly across the chords in the progression. For example, 12 bars with a 4-chord progression gives 3 bars per chord. With 4 beats per bar, that's 12 beats of each chord.

progression (array of strings, required) — Chord progression as Roman numerals. The engine resolves each numeral to a fully voiced chord in the theme's key and mode. Available numerals: I through VII (major) and i through vii (minor). Quality suffixes can be appended: "iim7", "Vmaj9", "viidim", "IVaug", etc.

The chords cycle through the bars evenly. A 4-chord progression over 12 bars means each chord lasts 3 bars. A 5-chord progression over 10 bars means each chord lasts 2 bars. Non-integer ratios are handled gracefully.

density (string, required) — Controls the harmonic richness and rhythmic activity across all voices.

- **"sparse"** — Triads only (3 notes per chord). Harmony uses whole notes (one chord strike per chord change). Melody uses free-form rhythm with varied durations. Bass depends on style. Very open, ambient, meditative.
- **"medium"** — Seventh chords (4 notes). Harmony uses half notes. Melody uses sparse eighth notes with rests for breathing. More melodic motion while retaining space.
- **"full"** — Extended chords up to elevenths (up to 6 notes). Harmony uses quarter notes. Melody uses eighth notes with moderate rests. The richest, most active texture.

melody (string, required) — The melodic behavior algorithm for this section.

- **"generative"** — Freely picks notes from a weighted pool of chord tones (50%) and scale tones (45%), with a slight chromatic allowance (5%). Prefers notes within a fifth of the current pitch for smooth motion. Good for ambient wandering, atmospheric texture, and sections that shouldn't draw attention to the melody.
- **"lyrical"** — Stepwise motion, gravitating toward chord tones. Prefers scale tones within 3 semitones of the current pitch, weighted toward chord tones within a fifth. Has a directional bias to avoid pure random walk. Produces the most singable, melodic output. Best for sections where you want a clear melody line to play over or complement.

- **"sparse"** — Wide intervals, few notes, lots of space. Only plays about 40% of available rhythm slots; the rest are rests. When it does play, it chooses chord tones anywhere in the register, producing large leaps. Slightly softer velocity. Best for atmospheric openings, transitions, and fade-outs.
- **"develop"** — Applies motif transforms to generate melodic material. Picks a random transform from the motif's transform_pool, applies it to the interval and rhythm sequences, then maps the result onto scale and chord tones starting from the nearest chord tone to the previous pitch. When the transformed motif is exhausted, remaining rhythm slots are filled generatively. Falls back to generative if no motif is defined. Best for climactic sections, emotional peaks, and anywhere you want the motif to be heard in transformed states.

bass_style (string, required) — The bass line algorithm.

- **"root_only"** — One root note per chord, held for the full chord duration. Minimal, drone-like. The simplest possible bass. Good for openings, closings, and sparse textures.
- **"root_fifth"** — Alternates root and fifth within each chord's duration. The root gets the first half, the fifth gets the second half (slightly softer). Classic new age and ambient bass movement. Provides gentle harmonic reinforcement without being busy.
- **"walking"** — Stepwise chromatic movement between chord roots. Each beat gets a note, starting on the current chord's root and stepping toward the next chord's root. The first beat of each chord is accented. Provides forward motion and melodic interest in the bass. Best for warmer, more active sections.
- **"pulse"** — Repeated root notes on every beat (quarter notes by default). The first beat is accented, subsequent beats are softer. Rhythmic and driving. Best for sections that need momentum.
- **"pedal"** — Holds a single pedal tone (the tonic of the first chord) regardless of what the harmony is doing. Very Eno-ish. The harmony moves above while the bass stays rooted. Creates tension when the harmony moves away from the tonic and resolution when it returns. Best for sections with a drone quality or where you want harmonic ambiguity.

arc (string, required) — The velocity shape applied across the section. Controls the dynamic contour — how loud or soft the section feels over time.

- **"flat"** — Constant velocity throughout. No dynamic shaping. Useful for very short sections or mechanical textures.
- **"swell"** — Crescendo to the midpoint, decrescendo back. The classic emotional arc: build to a peak, then release. The midpoint reaches about 130% of base velocity; the ends are about 70%. Best for sections with a clear emotional peak.

- **"fade_in"** — Starts at 50% velocity, builds linearly to full. The section emerges gradually from silence. Best for openings and entrances.
- **"fade_out"** — Starts at full velocity, fades linearly to 50%. The section dissolves. Best for closings and transitions to silence.
- **"breath"** — Two gentle sinusoidal swells across the section. Ranges from 75% to 100% of base velocity. Organic, like breathing. Best for long sustained sections where you want movement without a single dramatic peak. Also controls prosody: when used as the arc, it sets the tension ceiling to COLOR (ninth, thirteenth).

beats_per_bar (integer, optional, default: 4) — The time signature numerator. The engine writes a 4/4 time signature in the MIDI metadata regardless of this value, but it uses this number internally to calculate beat counts, rhythm grids, and bar divisions. Setting this to 3 gives waltz-time feel; setting it to 6 gives compound time.

counterpoint (object, optional) — If present, a counterpoint voice is generated against the melody in this section. If absent, no counterpoint is generated.

- **counterpoint.species** (string) — "first" for strict note-against-note counterpoint (every interval consonant, no rests), or "free" for mixed-rhythm counterpoint with passing tones and occasional rests.
- **counterpoint.register** (string) — "below" places the counterpoint voice below the melody (C3-A4 range), "above" places it above (G4-E6 range). "below" is typical — it creates a warm foundation. "above" creates a descant effect.
- **counterpoint.dissonance** (string) — Controls dissonance handling in free species. "none" means consonances only (like first species but with free rhythm). "passing" allows dissonance on weak beats as passing tones (stepwise approach only). "free" allows dissonance with resolution — the most expressive option. Ignored for first species.
- **counterpoint.velocity** (integer, optional) — Base MIDI velocity for counterpoint notes. Typically lower than melody (50-60) so the counterpoint supports rather than competes with the melody and any live performance.

notes (string, optional) — Section-specific notes. Not used by the engine.

---

9. Building a Piece Step by Step

Step 1: Choose a Key and Mode

Start with the emotional character you want.

For warm, romantic, resolved pieces: ionian (major). G ionian is an excellent default — it's open and resonant, particularly under guitar.

For darker, atmospheric pieces with warmth: dorian. D dorian is the classic choice — it's the key of most ambient and new age instrumental music.

For floating, luminous pieces: lydian. F lydian gives you the I-II shimmer.

For melancholic, darker pieces: aeolian (natural minor).

Step 2: Define a Tempo Range

Ambient music typically lives between 50 and 80 BPM. Slower tempos (48-60) feel meditative and spacious. Mid-tempos (60-75) have gentle forward motion. Faster tempos (75-90) feel more like film score than ambient.

Set the range wide enough to accommodate different pieces from the same theme.

Step 3: Create a Motif (or Phrase)

You have two options:

Option A — Hand-crafted motif: Write an intervals array and rhythm array directly. Think of a short melody and transcribe its semitone movement. A good starting approach: hum or sing a 4-6 note phrase, then write down how many semitones each note moves from the previous one (positive = up, negative = down).

Option B — Prosody phrase: Set the "phrase" field to a word or short phrase that captures the emotional seed of the piece. "Rebecca" becomes a 3-syllable motif (short-LONG-short) with harmonic awareness. "I have loved you" becomes a 4-syllable motif with different stress patterns. The prosody engine handles the rest.

Prosody is best for pieces named after people, places, or phrases where the word's natural rhythm should shape the music. Hand-crafted motifs give you more precise control.

Step 4: Write Sections

Start with two voices and one section before adding complexity. A minimal viable piece:

```
json
```

```json
{
  "piece": {
    "title": "Test",
    "tempo": 65,
    "sections": [
      {
        "name": "opening",
        "bars": 8,
        "progression": ["I", "V", "vi", "IV"],
        "density": "sparse",
        "melody": "generative",
        "bass_style": "pedal",
        "arc": "fade_in"
      }
    ]
  }
}
```

Generate this, listen, then add sections. Build the emotional arc by changing density, melody behavior, and bass style across sections.

Step 5: Shape the Emotional Arc

A typical arc for a 5-10 minute piece:

1. **Opening** — sparse density, sparse or generative melody, pedal bass, fade_in arc. Establish the key and mode with minimal material.
2. **Growth** — medium density, lyrical melody, root_fifth bass, swell arc. The piece finds its voice. Add counterpoint here if desired.
3. **Peak** — full density, develop melody, walking bass, swell arc. The emotional center. The motif is most audible here through development.
4. **Breath** — sparse density, generative melody, pedal bass, breath arc. A moment of stillness after the peak.
5. **Return/Close** — medium to sparse density, lyrical then sparse melody, root_fifth then root_only bass, swell then fade_out arc. Memory of what came before, dissolving into silence.

Step 6: Generate and Listen

```bash
```

```
cd forma
python main.py ../compositions/your_theme.json ../compositions/your_piece.json
```

Import the MIDI into Logic Pro. Assign instruments. Listen through once without playing over it. Note what works and what doesn't. Then iterate: adjust progressions, swap bass styles, change density in specific sections, regenerate.

---

10. DAW Integration (Logic Pro)

Importing MIDI

File → Import → MIDI File (or drag the .mid file directly onto the Tracks area). Logic creates one track per MIDI track. The track names (Melody, Counterpoint, Harmony, Bass) appear automatically.

Assigning Instruments

Each track arrives as a generic MIDI instrument. Click the instrument slot on each track and load your preferred plugin. The Arturia V Collection is the primary target — recommended starting points:

Track	Suggested Instruments
Harmony	Mellotron V (strings), Solina V, JUP-8000 V
Melody	Pigments (warm pad), Analog Lab, CMI V (vibes)
Bass	Mini Moog V4, JUP-8000 V
Counterpoint	Solina V, String Studio VS-3

Performance Workflow

The generated MIDI is the bed — the harmonic and rhythmic foundation. Your instrument (guitar, keyboard, voice) is the foreground. In practice:

1. Set Logic to loop the section you want to practice over
2. Play freely over the bed, finding melodic ideas that respond to the harmony
3. When you find something you like, record a take
4. Comp multiple takes into a final performance

5. The generative tracks provide a bed that changes subtly each time you regenerate, keeping the collaboration between you and the engine fresh

Mixing Notes

The counterpoint voice should be significantly quieter than the melody — it's a supporting voice, not a lead. Pull its velocity down in the composition JSON (50-55 is typical) and consider further attenuation in the DAW mix. The harmony track is the emotional center; it should be warm and present but not loud. The bass should be felt more than heard in ambient contexts.

---

11. Prosody Details

How Phrase Analysis Works

1. The phrase is split into words
2. Each word is looked up in the CMU Pronouncing Dictionary (if the `pronouncing` library is installed)
3. For words found in the dictionary, phonetic stress patterns are extracted (0 = unstressed, 1 = primary stress, 2 = secondary stress), along with vowel identity for duration mapping
4. For words not in the dictionary, a heuristic fallback counts vowel groups to estimate syllable count and applies a default stress pattern (stress on the first syllable for 2-3 syllable words, stress on the second syllable for 4+ syllable words)

Prosody in Theme JSON

```json
{
  "theme": {
    "name": "Rebecca Prosody",
    "key": "G",
    "mode": "ionian",
    "tempo": { "min": 48, "max": 66 },
    "phrase": "Rebecca"
  }
}
```

When the engine encounters a theme with "phrase" but no "motif," it calls the prosody engine once per section, passing the section's arc, melody, density, the first chord of the progression, and the theme's key and mode. Each section gets a unique motif shaped by its emotional context.

Harmonic Mode vs Fallback Mode

With full harmonic context (key + mode + chord + section), the prosody engine builds a note pool organized by tension level, selects notes using the section's tension profile, and enforces voice leading. This is the primary mode.

Without harmonic context (just a phrase, no key/mode/chord), the engine falls back to an interval-based mode that maps stress patterns to interval pools: primary stress maps to larger intervals ([3, 4, 5] semitones), secondary stress to moderate intervals ([1, 2]), and unstressed syllables to small movements ([-1, 0, 1]). Directional contour is computed from the stress peak.

Rhythm from Prosody

Note durations are derived from stress and vowel identity:

- Primary stress (1): longer notes (2.0-3.0 beats in slow mode, 1.5-2.0 in normal)
- Secondary stress (2): medium notes (1.0-1.5 beats)
- Unstressed (0): short notes (0.5-1.0 beats)
- Long vowels (EY, IY, OW, UW, AY, AW, OY): duration multiplied by 1.25

---

12. Validation and Warnings

The engine validates both theme and piece before generation. Hard errors prevent generation; warnings are printed but generation proceeds.

Theme Validation

- key is required
- mode is required and must be a valid mode name
- tempo is required and must have min and max keys; min must be ≤ max
- If both motif and phrase are present, a warning is issued (motif takes precedence)

Piece Validation

- sections must be present and non-empty
- Each section must have a progression (non-empty array) and bars
- density, if present, must be one of: sparse, medium, full
- melody, if present, must be one of: generative, lyrical, sparse, develop
- bass_style, if present, must be one of: root_only, root_fifth, walking, pulse, pedal
- arc, if present, must be one of: flat, swell, fade_in, fade_out, breath

- tempo, if present, is checked against the theme's range (warning if outside, not an error)

---

13. Tips and Patterns

Start small. Two voices and one section before adding complexity. You'll hear problems faster in a minimal piece.

Iterate by ear before cutting code. Listen to a generated piece several times before deciding to change parameters. What sounds wrong on first listen sometimes reveals itself as interesting on the third.

The progression carries more emotional weight than any other parameter. A I-V-vi-IV in ionian will always sound warm and familiar. A i-VII-iv-v in dorian will always sound moody and atmospheric. Start with the progression, then shape the other parameters around it.

Density and melody behavior are the main texture controls. The difference between a sparse/sparse section and a full/develop section is enormous. Use this range deliberately across your sections to create contrast.

Pedal bass creates the most ambient texture. The tonic drone underneath shifting harmony is the signature of Eno-style generative music. Use it for openings, closings, and still moments.

Walking bass creates the most forward motion. Use it for peaks and climactic sections where you want the piece to feel like it's going somewhere.

Counterpoint should be rare and quiet. It's most effective in the middle sections of a piece — when the texture has enough other voices to support a second melodic line. Keep the velocity low (50-55) and use "below" register so it doesn't compete with your live performance.

Don't use counterpoint in sparse sections. There's not enough melodic material for the counterpoint engine to work against. It sounds thin and mechanical.

Breath arc for long sections, swell for emotional peaks. Breath gives two gentle waves across the section — good for 20+ bar sections that need to feel alive without a single dramatic peak. Swell builds to a clear midpoint climax and is best for sections with a defined emotional center.

The same theme can produce very different pieces. The Rebecca theme (G ionian, singable motif) can produce a 3-minute sketch with 3 sections or a 10-minute romantic piece with 7 sections. The theme is the identity; the composition is the architecture.